Security

NestUp

Authentication, authorization, secrets, and remaining risks

Internet Technologies — Become a Full-Stack Engineer

RUNI CS 2026 · Final project

Live product **nestup-kappa.vercel.app**

Repository **github.com/nestupweb/nestup**

Posture: 21 tables, **RLS enabled on every one of them**, 72 policies · authorization enforced in Postgres and not in application code

---

1. The governing principle

Nearly every decision described in this document follows from a single principle:

> **The application is not trusted to decide who may see what. Postgres is.**

Row Level Security means that the database itself refuses a query for rows which the caller is not permitted to read. Consequently, a bug in a page, in a Server Action or in a cache cannot expose the data of another member, because such a leak would first have to pass the database, and the database neither knows nor cares which line of TypeScript issued the query.

This principle matters more in this product than in most others, because NestUp holds telephone numbers, home addresses, private messages, and the times at which people have arranged to meet strangers.

---

2. Authentication

The mechanism is Supabase Auth, using email and password.

- **Email confirmation is mandatory.** This is a deliberate decision, which was maintained despite the temptation to remove it during development. In a product in which people eventually meet in real life, it is the least expensive barrier that exists against disposable accounts. Registration creates the row and mails a six-digit code, and the session is **withheld until that code is entered**. The test `tests/unit/signup-confirmation.test.ts` exists specifically in order to prevent this behavior from regressing.
- **Passwords are never handled by us.** Supabase Auth hashes them and stores them, and the application never sees, logs or stores a password.
- **Sessions are httpOnly cookies**, managed by `@supabase/ssr` and refreshed by the edge proxy on every request.
- **Password recovery** is performed through an emailed link which leads to `/auth/confirm?type=recovery` and then to `/reset-password`.
- **Changing an email address or a password requires re-authentication** using the current password.
- **Authentication email is rate-limited** by the `auth_mail_throttle` table, so that the endpoint cannot be used in order to spam an address.

Signing out is a security operation and is treated as one

Signing out posts to `/auth/signout`, which is a Route Handler that terminates the session and answers with **HTTP 303**.

The 303 status is the essential point. A `redirect()` issued from a Server Action performs a *soft* navigation, meaning that the React client survives it, and so does everything held in memory. This includes the `"use cache: private"` entries of that browser, namely the deck, the inbox and the profile tabs, as well as the cache of already-rendered pages maintained by the router. As a result the member was shown a signed-out page while his or her own data remained sitting behind it, one press of the back button away, and **the next person who signed in on that computer inherited the tab in exactly that state.**

A 303 status forces a genuine document navigation, which tears the client down and rebuilds it empty. Nothing in `next/cache` is able to reach into the browser memory belonging to another session; only a reload can accomplish this.

The handler is **POST-only**, so that no third-party page is able to sign a member out by means of an `<img>` tag. Two tests maintain this property: `signout-clears-cache.test.ts` asserts both the 303 status and the absence of a GET handler, and `member-actions.test.tsx` asserts that the button posts to the route rather than calling an action. A refactor back into an action would appear to be a harmless tidying-up operation and would silently restore the leak.

This was verified on production: after signing out, the JavaScript context is destroyed, `/swipe` redirects to `/login`, the back button cannot restore a signed-in page, and a second member who signs in using the same tab sees only his or her own profile.

---

3. Authorization

There are four layers, and each of them assumes that the layers above it may fail.

Layer 1 — the edge proxy (`proxy.ts`)

This layer runs before any page renders. It refreshes the session and redirects unauthenticated requests away from `/swipe`, `/matches`, `/listing`, `/profile`, `/chat` and `/browse/[id]/chat`.

It deliberately covers **page routes only**, because an API caller must receive a JSON 401 response and not an HTML redirect to a login page. API routes therefore verify authorization inside the handler itself.

Layer 2 — the page

Every signed-in page calls a gate, either `requireUser()` or `requireCachedProfile()`, which redirects to `/login` when there is no session and to `/login?error=suspended` when the account is suspended.

Layer 3 — the Server Action

Every mutation begins with `requireUser()`, which is **uncached and is executed on every write**. Section 8 explains why this distinction is drawn deliberately.

Layer 4 — Row Level Security

This is the layer which actually matters. It consists of 72 policies distributed across 21 tables. The following are examples of the shapes which are used:

RULE	HOW IT IS EXPRESSED
A member may edit only his or her own profile	`using (user_id = auth.uid())`
Private details are accessible to the owner only	A separate `profile_details` table with an owner-only policy, together with a `public_profile_details()` function which returns only those fields that the owner chose to expose
Only participants may read the messages of a conversation	The policy joins through `conversations` and `listing_residents`
A blocked member disappears in both directions	The helper `blocked_user_ids()` is used inside the policies
The household of a room may edit its listing	`can_manage_listing()`, which admits the owner **or** a confirmed resident
Chats survive a deleted room	A second `listings` SELECT policy operating through `linked_to_listing()`, so that the history remains readable to the people who discussed that room

The pattern of having no write policy at all

Three tables have **no insert, update or delete policy whatsoever**:

TABLE	WRITTEN ONLY BY
`suspensions`	`apply_report_suspension()`
`listing_invites`	`invite_listing_roommates()`, `respond_to_listing_invite()`
`app_config`	SECURITY DEFINER functions only

This is the strongest pattern in the schema and is worth being able to explain. When a table must only ever be written in one specific manner, the correct approach is to **give it no write policy at all and exactly one `SECURITY DEFINER` function**. Under this arrangement there is no path whatsoever which leads to an incorrect write, as opposed to a policy which attempts to enumerate every possible incorrect write and might fail to include one of them.

The concrete consequence is that **a member cannot remove his or her own suspension.** This is true not because a policy forbids it, but because no route to that write exists in the first place.

Ownership which cannot be reassigned

A trigger named `listings_owner_is_permanent` refuses any ordinary `UPDATE` of `listings.owner_id`. The single legitimate case, which is the transfer of a shared listing to a roommate when an account is deleted, occurs inside `delete_own_account()`. That function sets a transaction-local GUC which nothing else in the schema sets. Therefore the listing survives its creator, and nevertheless there remains no way to steal one.

---

4. What requires a session

PUBLIC	REQUIRES A SESSION
`/browse`, which is the room list	The swipe deck
The page of a room, together with its photographs	Chat, and the initiation of a conversation
The map	Creating or editing a listing
Registration, login and password reset	The profile, and viewing the profile of another member
	Hearts, viewing history and scheduling a viewing
	Settings, reporting and blocking
	Anything under `/api/*` which touches member data

Browsing is open intentionally, because a marketplace which demands registration before it will display any supply cannot build the liquidity which it requires. Everything which reads or writes data belonging to a *person* requires a session.

---

5. Preventing access to the data of another member

Beyond RLS itself, there are three specific mechanisms.

Per-member cache keys. Every private cache entry is keyed and tagged using the identifier of the member, in the forms `deck:<userId>`, `profile:<userId>`, `saved:<userId>` and `chat:<userId>`. The test `cache-invalidation.test.ts` asserts that every per-member tag carries the identifier of the acting member, because a tag which omitted it would become **a single shared cache key for the entire application**, which is precisely the form that a cross-user leak takes.

Private caches never reach a shared store. Results produced under `"use cache: private"` live only in the memory of the requesting browser. The shared `"use cache"` mechanism is used for exactly one purpose, namely the public room list, and it operates through a deliberately **cookie-free** Supabase client, defined in `lib/supabase/public.ts` and marked `server-only`, so that its output cannot become member-specific.

Field-level control over exposure. The function `public_profile_details()` returns only those fields which a member chose to make visible, and contact details are hidden by default. The separation between `profiles`, which is readable by members, and `profile_details`, which is accessible to the owner only, means that a newly added private column is private *by default* rather than being private because somebody remembered to make it so.

Storage. Chat photographs reside in a private bucket and are served as short-lived signed URLs. Furthermore, a photograph path must be located under the prefix of its own conversation, and this is verified on the server side.

---

6. Input validation

There are three layers, which are described in full in the Technical Design document:

1. **Client side**, which exists for convenience only and is assumed to be bypassable.
2. **Server side**, using six Zod schemas. The Server Action parses the input before touching the database, and the TypeScript types are inferred from the schemas so that the two cannot drift apart.
3. **Database level**, using `CHECK` constraints, enums, foreign keys and unique indexes.

The following specifics are relevant to security:

- **SQL injection is structurally absent.** All access is performed through PostgREST or through parameterized RPC calls. No SQL which is built through string concatenation exists anywhere in the codebase.
- **XSS.** React escapes output by default, and there is no use of `dangerouslySetInnerHTML` anywhere in the product.
- **Open redirects.** The function `sanitizeNextPath()` guards every `?next=` parameter, so that an emailed parameter such as `?next=https://evil.example` cannot bounce a member off the site after login. This is covered by `redirect.test.ts`.
- **Uploads.** The type and the size are checked on both sides, and a chat image path must belong to its own conversation.
- **Photo content.** An image is stored **only if** an AI check agrees that it shows what its slot claims that it shows. Therefore a bedroom slot cannot contain a picture of a car.
- **URL parameters** are parsed with `.catch()` fallbacks, so that hostile query strings produce default values rather than errors or unfiltered results.

---

7. Protecting API calls and secrets

API routes

Route Handlers authenticate inside the handler itself and return a JSON 401 response rather than performing a redirect. Every query beneath them still executes under RLS, and therefore an authenticated caller who experiments with the identifier of another member receives an empty result rather than data.

The route `/api/places` exists specifically **so that the browser never calls Overpass directly**. Consequently the upstream call, together with any key or quota which may apply to it in the future, remains on the server side.

Secrets

SECRET	WHERE IT LIVES	DOES IT REACH THE BROWSER?
`NEXT_PUBLIC_SUPABASE_URL`	Environment variable and build argument	Yes, and it is designed to be public
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Environment variable and build argument	Yes. It is public by design, it is powerless without a session, and it is governed by RLS
`SUPABASE_SERVICE_ROLE_KEY`	Vercel **sensitive** production environment variable	**Never**
`GEMINI_API_KEY`	Vercel sensitive environment variable	**Never**
`SMTP_*` credentials	Vercel sensitive environment variables	**Never**
`GOOGLE_CLIENT_SECRET`	Vercel sensitive environment variable	**Never**
`SUPABASE_ACCESS_TOKEN`	Local machine only, for the auth-config script	**Never**

The following rules are actively followed rather than merely assumed:

- **The pattern `.env*.local` is gitignored**, and no environment file is tracked. The file `.env.example` documents the *names* of the variables and contains no values.
- **The service-role key is never passed as a build argument.** Build arguments are baked into the build output, whereas runtime secrets belong in runtime environment variables. This distinction is written into the deployment notes, because getting it wrong produces no visible symptom.
- **The fact that the anon key is public is not a weakness.** It identifies the project and grants nothing on its own, and every table which it is able to reach has RLS enabled.

- **A Stop hook commits automatically after every working session**, which turns "never place a secret inside a tracked file" into a standing rule rather than a habit.

---

8. A deliberate trade-off, disclosed

The identity read, meaning `auth.getUser()` together with the suspension lookup, is **cached per browser for the purpose of rendering**, using a window of 300 seconds.

The reason: the App Shell prerender advances through cached reads and stops at the first uncached read. That single round-trip was positioned at the top of every page and therefore kept everything behind it outside the prefetch, which cost approximately 300 ms of loading skeleton on every navigation between tabs.

What this costs: a suspension which is applied in the middle of a session now takes effect within the cache window rather than on the very next page load. The check itself is **not removed**, since `suspensions` is still read and both gates still redirect on the basis of it. It simply refreshes at most once per window. This was raised with the product owner and was accepted explicitly.

What it does not cost, by design:

- The **edge proxy still calls `auth.getUser()` without caching on every request** to a protected route. That is the real gate, and it remains untouched. An expired or revoked session cannot reach these pages at all.
- **Every Server Action still uses the uncached `requireUser()`.** A cached identity is never what authorizes a write. The test `cached-session-boundary.test.ts` fails if any action inside `app/actions/` calls a cached reader. Both spellings compile and both appear to work, and therefore only a test is capable of enforcing this boundary.
- **RLS is entirely unaffected.** Every query still executes on the cookie-bearing client, and therefore Postgres decides what is returned regardless of what the cache believes.

Disclosing this trade-off is itself the point. A security document which lists only strengths is not a security document.

---

9. Remaining risks

These are ordered according to how much they would actually matter.

1. **There is no identity verification.** Only email confirmation is performed. Anybody is able to create an account using a working address, and members subsequently meet in person. This is the largest real-world risk in the product, and it cannot be solved by code alone, since it requires identity verification together with a policy describing what happens when that verification fails.
2. **Suspension latency**, of up to five minutes, as described in §8.

3. **There is no rate limiting on most writes.** Only authentication email is throttled. Message sending, listing creation and report submission are protected by RLS but not by any volume limit, and therefore a determined authenticated member could send spam.
4. **There is no CSRF token on the sign-out form.** This is mitigated by the handler being POST-only and by the session cookies using `SameSite=Lax`, which means that they do not travel on a cross-site POST. The severity is low, since the worst possible outcome is being signed out, but it is nevertheless a gap.
5. **Photo moderation relies on a single model.** The Gemini check is a check of the content *type* and not a safety check. It confirms that a bedroom photograph shows a bedroom, but it is not a general filter against abuse.
6. **There is no audit log.** No record exists of who read what, and therefore the scope of a breach could not be determined after the fact.
7. **Message content is not encrypted at rest** beyond the disk encryption provided by Supabase. Anybody who has access to the database is able to read the chats.
8. **There is no two-factor authentication.**
9. **Reports are not acted upon automatically.** The function `apply_report_suspension()` exists, but nothing calls it upon reaching a threshold. A moderator would have to act, and there is no moderator.
10. **There is dependency risk.** No automated vulnerability scanning is performed in CI.

---

10. What a hardened version would add

The following list is presented in order of priority:

1. **Rate limiting** on writes, applied per member and per IP address, at the edge.
2. **A real moderation queue**, in which reports are surfaced to a human being, with `apply_report_suspension()` connected to a threshold.
3. **Two-factor authentication**, at least as an option, for accounts which share contact details.
4. **An audit log** covering reads of sensitive fields, so that the scope of a breach can be determined.
5. **CSRF tokens** on form posts which change state, rather than relying on SameSite alone.
6. **Automated dependency scanning**, meaning Dependabot together with `npm audit` in CI.
7. **A Content Security Policy**, which is currently absent and which would provide hardening against injected script even though React escapes output by default.
8. **Shorter session lifetimes**, combined with silent refresh.
9. **A tested backup and restore procedure.** Supabase performs backups, but nobody has ever restored one, and an untested backup is a hope rather than a plan.
10. **RLS policy tests** which execute real queries as two different members against a dedicated test project. This is the one layer which the unit suite is unable to reach at present.
